

RISKFORGE-QRM

Quantitative Market Risk Modelling, Validation & Reverse Stress Testing Framework

Project Report

Reference Period: 2015-01-01 to 2024-12-31 | 5 Risk Factors | 13,045 Raw Price Rows | 2,358 VaR Forecasts per Model

Methods: Historical / Parametric / EWMA VaR · Expected Shortfall · Kupiec & Christoffersen Backtesting · Stress Testing ·
Reverse Stress Optimization · Risk Attribution

August 2026

Table of Contents

1. Project at a Glance
2. Problem, Objective and Scope
3. Dataset and Data Generation
4. System Architecture
5. The Complete Pipeline, Phase by Phase
6. Statistics and Quantitative-Risk Concepts
7. Formula Sheet and Derivations
8. Codebase Walkthrough
9. Reference Results and Interpretation
10. Streamlit Dashboard Deep Dive
11. Testing, QA and Reproducibility
12. Limitations and Honest Caveats
13. Interview Defense Guide
14. Glossary
15. Quick Revision Sheet
16. Conclusion and Future Work

1. Project at a Glance

1.1 What is this project, really?

RiskForge-QRM is an educational, industry-style quantitative market-risk framework. It takes a portfolio built from five market risk factors, estimates how much that portfolio could lose over one day using three competing statistical models, checks whether those loss estimates actually behaved reasonably against what really happened, tests the portfolio against severe hypothetical scenarios, works backwards to find what shocks would be needed to cause a target loss, and explains which risk factors are driving the portfolio's risk.

It is not a black-box predictor and not a production bank risk engine. Every formula, every design decision, and every limitation is meant to be explainable on a whiteboard.

EXPLAIN LIKE I'M 10

Imagine you own a small basket of five things — some stocks, a bit tied to interest rates, some foreign currency, some oil, some gold. Every day the value of your basket wobbles up and down. This project builds a careful, tested system that asks: 'How much could I lose tomorrow?', checks whether that guess has been right in the past, imagines really bad days, and explains which part of the basket is causing most of the wobble.

1.2 The project in one sentence

"I built a fixed-weight market-risk portfolio, estimated one-day 99% VaR using three statistical approaches, backtested those forecasts, stress-tested the portfolio, solved a reverse-stress problem, and attributed portfolio risk back to individual risk factors."

1.3 The full workflow

Market Data → Data Quality → Portfolio Construction → Historical VaR → Parametric VaR → EWMA VaR → Expected Shortfall → Backtesting → Stress Testing → Reverse Stress Testing → Risk Attribution → Model Validation Report → Streamlit Dashboard.

1.4 What the project actually does, step by step

1. Generates or ingests risk-factor price data (synthetic by default; yfinance or CSV loaders are also implemented for real data).
2. Runs data-quality checks: duplicates, missing values, date gaps, stale values, outlier returns.
3. Builds a fixed-weight portfolio return and P&L series from five risk factors.
4. Fits three competing 1-day VaR models — Historical, Parametric (variance-covariance), and EWMA-volatility — plus historical Expected Shortfall, all as rolling, strictly out-of-sample forecasts with no look-ahead bias.
5. Backtests every model: exception rate, Kupiec proportion-of-failures test, Christoffersen independence test, and model comparison.
6. Runs historical, hypothetical, and sensitivity stress tests across four defined scenarios.
7. Solves a reverse stress test — the smallest combination of market shocks that would cause a 10% portfolio loss — as a constrained optimization problem, with independent verification and a multi-start stability check.
8. Runs risk attribution: variance decomposition and scenario factor ranking.

9. Writes reproducible CSV tables, figures, and a structured model-validation report.
10. Presents everything through an interactive Streamlit dashboard with seven guided pages.

1.5 Why this project exists (the UBS connection)

This project was built in preparation for a UBS Quantitative Risk Modelling (QRM) intern role. The actual job description centers on credit-exposure measures for the banking and trading book. RiskForge-QRM is instead a market-risk framework (VaR / ES / backtesting / stress). It demonstrates the same underlying skill set — statistics, model validation, Python engineering, and SQL awareness — that the role asks for, but it does not attempt counterparty or credit-exposure modelling such as PFE or EAD. This distinction is worth stating explicitly and proactively in an interview.

INTERVIEW ANGLE

If asked “why isn't this a credit-risk project if you're interviewing for a credit-exposure role?”, say: “I built a market-risk framework because it let me demonstrate the full model-build-to-validation lifecycle end to end. The statistical skills — quantile estimation, hypothesis testing, scenario analysis, model validation — transfer directly, even though credit exposure (PFE/EAD) is a different modelling problem I'd approach differently.”

2. Problem, Objective and Scope

2.1 The real-world problem

A financial portfolio is exposed to several market variables that move every day. A risk team needs a repeatable way to estimate potential short-horizon loss, compare different modelling assumptions, check whether the forecasts behave reasonably, understand severe scenarios, and understand which risk factors are responsible for the portfolio's risk. RiskForge-QRM demonstrates this entire workflow end to end.

2.2 Main objective

Estimate and validate 1-day portfolio VaR and Expected Shortfall across three competing model families, then stress-test and reverse-stress the same portfolio.

2.3 Secondary objectives

Objective	Goal
A — Data quality	Make sure the input data is structurally usable
B — Portfolio construction	Turn multiple factor movements into one portfolio return series
C — Model comparison	Compare Historical, Parametric and EWMA VaR
D — Statistical validation	Use exception behaviour and formal backtests
E — Scenario analysis	Measure losses under severe hypothetical conditions
F — Reverse stress	Find the shocks needed to reach a target loss
G — Explainability	Attribute risk to individual factors
H — Reproducibility	Make the same run produce the same outputs

2.4 What is deliberately out of scope

The project does not claim to be a production bank risk engine, a regulatory capital calculator, a Basel compliance implementation, an options or non-linear pricing engine, a real-time market-data platform, or a counterparty-credit simulator. Stating these boundaries explicitly is a strength in an interview — it shows you know exactly what your model does and does not do.

RESEARCH NOTE

This project also does not calculate PFE, EAD, counterparty default probability, CVA, or Monte Carlo exposure profiles — all of which belong to credit-exposure modelling rather than market-risk modelling. Being explicit about this boundary is itself part of the project's discipline.

3. Dataset and Data Generation

3.1 The most important fact about the data

The reference run uses synthetic market data, not a real historical market dataset. This must be stated clearly and proactively in any interview discussion of this project.

EXPLAIN LIKE I'M 10

Instead of downloading real stock prices, the project has a little machine that invents realistic-looking price histories — with the same kinds of ups, downs, and jittery patterns real markets have — so the whole project can be built and tested without needing an internet connection.

3.2 Why synthetic data?

The project was designed to run offline and reproducibly. The synthetic generator recreates the exact same dataset from the same random seed (42), giving full repeatability. The project also contains real-data loaders — yfinance and local CSV — so the same architecture can later be run against real data without any redesign.

3.3 Reference configuration

Parameter	Value
Period	2015-01-01 to 2024-12-31
Risk factors	EQUITY_INDEX, RATES_10Y, FX_USD, OIL, GOLD
Portfolio weights	45% / 20% / 15% / 10% / 10%
Horizon	1 day
VaR confidence	99%
Rolling window	250 observations
EWMA lambda	0.94
Reverse-stress target	10% portfolio loss
Backtest significance level	5%
Raw price rows	13,045
VaR forecasts per model	2,358
Automated tests	36
Random seed	42

3.4 Risk factors and portfolio weights

Risk factor	Interpretation	Weight
EQUITY_INDEX	Broad equity-market movement	45%
RATES_10Y	10-year interest-rate / yield movement	20%
FX_USD	US-dollar / FX movement	15%

Risk factor	Interpretation	Weight
OIL	Oil-market movement	10%
GOLD	Gold-market movement	10%

3.5 How the synthetic returns are generated

The generator deliberately tries to look more realistic than simple random normal noise, using three ideas together:

- Fat tails — innovations are generated from a Student-t distribution with 6 degrees of freedom, making extreme observations more common than under a normal distribution.
- Volatility clustering — a simple GARCH-like recursion (persistence 0.90, shock weight 0.08) creates periods of relatively high and low volatility, so large movements today can raise tomorrow's volatility estimate.
- Cross-factor correlation — a correlation matrix is imposed via Cholesky decomposition, so the five factors are not independent and the portfolio has genuine diversification and covariance effects.

3.6 From returns to price levels

The generator first creates returns, then converts them into price/level series. For price-like factors, $P_t = P_{t-1} \times (1 + r_t)$. For RATES_10Y, the code instead treats the value as a rate level and accumulates yield changes, rather than compounding it like an asset price.

INTERVIEW ANGLE

Always flag the RATES_10Y simplification proactively: “I used a simplified linear rate-factor mapping because this is a market-risk modelling prototype. A real fixed-income portfolio would normally map a rate shock to price and P&L through duration, DV01, and convexity.”

3.7 Data quality checks

The pipeline checks for duplicates, missing or non-positive values, date gaps, stale repeated values, and extreme returns. The reference run reports Data Quality PASS = True, and the stored outlier-return table is empty.

RESEARCH NOTE

Implementation detail worth knowing: the overall PASS flag is based on duplicates, missing values, and flagged date gaps. Stale values and outlier returns are reported in their own table but do not independently flip the overall PASS flag to False. This is the kind of detail an interviewer checking your depth of understanding might probe.

3.8 The real-data path

The code supports `load_from_yfinance(...)` and `load_from_csv(...)`, the latter expecting columns date, risk_factor, value. The architecture is therefore not locked to synthetic data — swapping one line in the pipeline script would run the exact same modelling and validation logic against real market prices.

4. System Architecture

4.1 The big picture

Market or risk-factor data flows through a Data Loader (synthetic / yfinance / CSV), then a Data Quality layer (duplicates, missing values, date gaps, stale values, outliers), then Return Processing (price → returns), then the Portfolio Engine ($R_p = \sum w_i R_i$). From there it fans out into three parallel VaR models — Historical, Parametric, EWMA — which converge into Backtesting (exceptions, Kupiec, Christoffersen). That feeds three further parallel analyses — Stress Testing, Reverse Stress, and Risk Attribution — which converge into a Reporting Layer (CSV + figures + validation report) and finally the Streamlit Dashboard presentation layer.

A parallel quality layer (tests/) provides unit tests across data, portfolio, models, validation, stress, reverse-stress and attribution. A future storage layer (sql/) contains a PostgreSQL schema and example queries, prepared but not yet wired into the pipeline.

4.2 The core design principle: separation of responsibilities

Layer	Responsibility
Data code	Loads and validates data
Model code	Calculates risk (VaR/ES)
Validation code	Tests risk forecasts (backtesting)
Stress code	Tests adverse scenarios
Attribution code	Explains risk
Reporting code	Saves outputs
Dashboard code	Displays outputs

This means the dashboard never secretly calculates the risk model. If the dashboard has a visual bug, the model code does not need to change. If the VaR model changes, the dashboard can still display its output. If real data replaces synthetic data, the modelling and validation layers can remain largely the same. That is a good industry-style separation of concerns.

INTERVIEW ANGLE

“I deliberately separated calculation from presentation. The Streamlit dashboard reads result CSVs — it never recomputes VaR itself. That means a dashboard bug can never silently corrupt a risk number.”

4.3 SQL architecture

The sql/ folder contains a PostgreSQL-style schema for market prices, risk-factor returns, portfolio positions, model configuration, risk results, backtest results, stress results, and reverse-stress results. Importantly, this SQL layer is prepared but not wired into the Python pipeline in the current project — it should never be described as already live.

4.4 Configuration architecture

configs/base.yaml stores risk factors, dates, weights, confidence level, rolling window, EWMA lambda, backtest significance, reverse-stress target, and shock bounds. configs/stress_scenarios.yaml stores the four scenario

definitions. This avoids scattering magic numbers throughout the Python code — every parameter used by the pipeline traces back to one readable configuration file.

5. The Complete Pipeline, Phase by Phase

Phase 1 — Data generation / loading

The pipeline loads synthetic data using `loader.load_from_synthetic(...)`. The output is a long-format table of factor levels, also saved into `data/raw/`.

Phase 2 — Data quality

The pipeline converts long data to wide factor levels, converts levels to returns, runs the data-quality checks described in Section 3.7, and stores the outlier-return table. The guiding principle: never blindly feed bad data into a risk model.

Phase 3 — Portfolio construction

The portfolio return is $R_{p,t} = \sum w_i R_{i,t}$. For example, if the equity return is -10% and the equity weight is 45%, equity contributes $0.45 \times -10\% = -4.5\%$ to the portfolio return. The same calculation runs for every factor and the contributions are summed. The code also performs a small, hand-checkable manual verification example, used directly in the test suite.

Phase 4 — Risk models

- Historical VaR — for every forecast date, take the previous 250 returns, calculate the 1st percentile, and report it as a positive loss (multiply by -1). No future return is used.
- Parametric VaR — for every forecast date, calculate the mean and standard deviation of the trailing window, obtain the 1% normal quantile, and convert the lower-tail return into a positive loss.
- EWMA VaR — for every forecast date, update variance recursively using only historical observations ($\lambda = 0.94$), calculate current volatility, use the trailing 250-day mean, and apply the normal-tail VaR formula.

Phase 5 — Backtesting

For each model: align the VaR forecast with the return that actually happened, calculate realised loss = -return, mark an exception whenever realised loss exceeds VaR, calculate the exception rate, run the Kupiec test, run the Christoffersen test, and combine the two into an overall validation verdict.

Phase 6 — Stress testing

Four configured scenarios (EQUITY_CRASH_2008_STYLE, RATES_SHOCK_UP, OIL_SUPPLY_SHOCK, FX_CRISIS) each define shocks for all five factors. Portfolio loss uses the same linear mapping: Portfolio return = $\sum w_i \times \text{shock}_i$, and Portfolio loss = -portfolio return. The project also runs a one-factor equity sensitivity analysis.

Phase 7 — Reverse stress

Target: a 10% portfolio loss. The optimizer finds a shock vector that reaches at least 10% loss, stays within configured bounds, and has a small L2 norm. Formally: minimize $\sum x_i^2$ subject to $-\sum w_i x_i \geq 0.10$ and $\text{lower}_i \leq x_i \leq \text{upper}_i$. The solution is independently checked and the optimizer is restarted five times to test stability.

Phase 8 — Risk attribution

Two views are calculated. Variance attribution: using covariance matrix Σ , portfolio variance = $w'\Sigma w$, and factor i 's contribution is $w_i(\Sigma w)_i$ / portfolio variance. Scenario attribution: for each stress scenario, factor contribution = weight $\times$ shock, and the project also averages absolute factor contributions across scenarios.

Phase 11 — Reporting

The pipeline saves the model comparison, statistical tests, backtest tables, stress results, reverse-stress results, sensitivity results, attribution results, charts, and the structured model validation report. This stage is labelled Phase 11 in the codebase because it follows the broader project blueprint — there are no separate Phase 9 and Phase 10 calculations in the current implementation, which is a detail worth knowing rather than hiding if asked.

Phase 12 — Dashboard

The Streamlit app reads the generated results and presents them visually. It is a presentation layer, not the modelling engine.

6. Statistics and Quantitative-Risk Concepts

6.1 Return, mean, volatility, covariance

For a price P , the simple return is $r_t = P_t / P_{t-1} - 1$. Mean return gives the average return in the sample and is used by both the parametric and EWMA models. Standard deviation (volatility) measures how spread out returns are — higher volatility means more uncertainty around normal portfolio movement. Covariance tells us whether two factors tend to move together; because portfolio risk is not simply the sum of individual risks, covariance is exactly where diversification comes from.

EXPLAIN LIKE I'M 10

If your two friends always get happy or sad on the same days, that's like high covariance. If one is happy exactly when the other is sad, that's negative covariance — and having both of them around actually smooths out your day. Diversification in a portfolio works the same way.

6.2 Value at Risk (VaR)

At confidence c , $\text{VaR}_c = -Q_{1-c}(R)$. For 99% confidence, $\text{VaR}_{99} = -Q_{0.01}(R)$. VaR answers a threshold question: at a chosen confidence level, what loss level should not normally be exceeded on one day? It does not tell us the average loss beyond that threshold — that is what Expected Shortfall is for. VaR is a threshold, not the worst possible loss, and that distinction is one of the most important things to get right in an interview.

6.3 Expected Shortfall (ES)

Expected Shortfall looks beyond VaR. Historical $\text{ES}^c = -E[R \mid R \leq Q_{1-c}]$: if we are already in the worst 1% tail, how large is the average loss? The project implements historical ES and a normal parametric ES function, and the pipeline calculates rolling historical ES — but the current reporting and dashboard layer does not persist or display a dedicated ES result table. This is an honest implementation gap worth naming directly rather than glossing over.

6.4 EWMA volatility

EWMA stands for Exponentially Weighted Moving Average. The variance recursion is $\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda)r_{t-1}^2$. With $\lambda = 0.94$, most weight stays with the previous volatility estimate while 6% weight is assigned to the latest squared return — so a large recent return increases the volatility estimate quickly.

6.5 Backtesting, exceptions, and the two formal tests

Backtesting means taking a model's old forecasts and comparing them with what actually happened — a model-validation idea, distinct from training a machine-learning model. An exception occurs when realised loss $>$ VaR forecast; at 99% VaR the expected exception rate is approximately 1%.

Test	Null hypothesis	What it checks
Kupiec proportion-of-failures	True exception probability = target exception probability	Whether the total exception count is statistically consistent with the target rate
Christoffersen independence	Exceptions occur independently over time	Whether exceptions cluster together rather than scattering randomly

A model can have an acceptable total exception rate but still have clustered exceptions — which is exactly why both tests are run together rather than relying on exception count alone.

6.6 Stress testing vs. reverse stress testing

Stress testing is scenario analysis: it does not ask what is the statistically expected loss, but what happens if a severe predefined event occurs. Reverse stress testing turns the question around — instead of starting with a shock and finding the loss, it starts with a target loss (10%) and searches for a shock vector sufficient to reach it. This is an optimization problem rather than a simulation.

6.7 Risk attribution and model risk

Risk attribution answers which factor explains the portfolio's risk, using covariance-based variance decomposition plus a ranking of factor contributions across stress scenarios. Model risk means the model can be wrong because assumptions, data, or implementation can be wrong. This project addresses model risk by comparing multiple VaR models, backtesting, unit testing, documenting explicit assumptions, stress testing, independently verifying the reverse-stress solution, and clearly documenting limitations — arguably the single strongest interview point in the whole project.

7. Formula Sheet and Derivations

7.1 Portfolio return

With weights $w = [0.45, 0.20, 0.15, 0.10, 0.10]$ and factor returns $r = [r_1, r_2, r_3, r_4, r_5]$:

$$R_p = 0.45r_1 + 0.20r_2 + 0.15r_3 + 0.10r_4 + 0.10r_5$$

This is simply a weighted average of the factor returns.

7.2 Historical VaR

Sort the historical returns from worst to best. At 99% confidence, find the 1st percentile. If the percentile is -2.0% , $\text{VaR} = -(-2.0\%) = 2.0\%$ — reported as a positive loss threshold.

7.3 Parametric VaR

Assuming $R \sim N(\mu, \sigma^2)$, the 1% normal quantile at 99% confidence is approximately $z = -2.326$. The lower-tail return threshold is $\mu + z\sigma$, converted to a positive loss as $\text{VaR} = -(\mu + z\sigma)$. If $\mu = 0.05\%$ and $\sigma = 1\%$, $\text{VaR} \approx -[0.0005 - 2.326(0.01)] \approx 2.276\%$ — the exact project numbers change with every rolling window.

7.4 EWMA

Starting from an initial variance and recursively updating with $\lambda = 0.94$: $\sigma_t^2 = 0.94\sigma_{t-1}^2 + 0.06r_{t-1}^2$, then $\text{VaR}_t = -(\mu_t + z\sigma_t)$.

7.5 Historical Expected Shortfall

Find the VaR quantile q , take all returns worse than or equal to q , average them, and change the sign: $\text{ES} = -\text{mean}(\text{tail returns})$. Because ES is the average of the tail, it should generally be at least as large as VaR under the same convention.

7.6 Portfolio variance and risk contribution

With covariance matrix Σ : $\sigma_p^2 = w'\Sigma w$, and portfolio volatility $\sigma_p = \sqrt{w'\Sigma w}$. For factor i , $\text{Contribution}_i = w_i(\Sigma w)_i / \sigma_p^2$, and $\sum \text{Contribution}_i = 1$ (100%). A negative contribution can occur because covariance / diversification can reduce total portfolio variance.

7.7 Kupiec likelihood-ratio idea

With n forecasts, x exceptions, target probability p , and observed probability $\hat{p} = x/n$, the test compares the likelihood under the target probability p versus the observed probability $\hat{p}$. The likelihood-ratio statistic is approximately chi-square with 1 degree of freedom. You don't need to memorize every log-likelihood term for an interview — just the story: Kupiec tests whether the observed exception frequency is statistically consistent with the target frequency.

7.8 Christoffersen idea

Build a transition table counting consecutive-day pairs: no-exception→no-exception (n_{00}), no-exception→exception (n_{01}), exception→no-exception (n_{10}), exception→exception (n_{11}). The test compares an independence model

against one that allows transition probabilities to depend on the previous state — in short, it checks whether exceptions cluster.

7.9 Stress and reverse-stress formulas

For a shock vector x : $R_p(x) = \sum w_i x_i$, $\text{Loss}(x) = -R_p(x)$. Example: equity weight 45%, equity shock -20% → contribution to return = $0.45 \times -0.20 = -0.09$, i.e. equity contributes 9 percentage points of portfolio loss.

The reverse-stress problem is: minimize $\|x\|^2$ subject to $-w'x \geq \text{target_loss}$ and $\text{lower} \leq x \leq \text{upper}$. The L2 norm is minimized because the question is “what is a relatively small shock vector that is still sufficient to cause the target loss?” — not “what is the worst imaginable scenario?”

Because the loss equation is linear, the optimizer tends to use factors in proportion to their portfolio weights, subject to bounds. In the reference solution, the rate factor has a non-negative lower bound and ends up unused, while equity receives the largest negative shock because it carries the largest portfolio weight — a useful built-in sanity check on the optimizer's behaviour.

8. Codebase Walkthrough

8.1 Repository layout

Folder / file	Responsibility
configs/	All parameters (base.yaml, stress_scenarios.yaml) — nothing hardcoded in code
data/{raw,interim,processed}/	Data storage at each pipeline stage
sql/	PostgreSQL schema + data-quality/result queries (prepared, not yet wired in)
src/riskforge/data/	Loading (synthetic / yfinance / csv) + quality checks
src/riskforge/portfolio/	Risk-factor → portfolio return/P&L engine
src/riskforge/models/	Historical / parametric / EWMA VaR, Expected Shortfall
src/riskforge/validation/	Backtesting harness, Kupiec, Christoffersen
src/riskforge/stress/	Scenario engine + reverse-stress optimizer
src/riskforge/attribution/	Variance decomposition, scenario ranking
src/riskforge/reporting/	Tables, figures, validation-report writer
src/riskforge/utils/	Config loading, run metadata
scripts/run_pipeline.py	End-to-end orchestration — the best file to read first
tests/	36 pytest unit + manual-check tests
results/{tables,figures,reports}/	All generated outputs
dashboard/app.py	Optional Streamlit presentation layer

8.2 src/riskforge/data/

- loader.py — load_from_synthetic, load_from_yfinance, load_from_csv, save_raw.
- synthetic.py — creates reproducible synthetic market data: Student-t innovations, volatility clustering, correlation, business-day index, return-to-level conversion.
- quality.py — runs data-quality checks and converts long→wide and levels→returns.

8.3 src/riskforge/portfolio/engine.py

- validate_weights — checks that weights sum to 1.
- portfolio_returns — calculates $R_{p,t} = \sum w_i R_{i,t}$.
- portfolio_pnl — converts returns into P&L for a fixed notional.
- sample_covariance — calculates the sample covariance matrix.
- portfolio_variance — calculates $w' \Sigma w$.
- manual_check_example — a tiny hand-calculable example used directly in the test suite.

8.4 src/riskforge/models/

- historical_var.py — rolling empirical quantiles.

- `parametric_var.py` — mean, standard deviation, and a normal quantile.
- `ewma.py` — EWMA volatility and EWMA VaR, using $\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda)r_{t-1}^2$.
- `expected_shortfall.py` — historical ES, parametric ES, and rolling historical ES (calculated but not yet saved to a dedicated output file — see Section 12).

8.5 `src/riskforge/validation/`

- `backtest.py` — builds the date-level backtest table: realised loss, exception flag, exception rate, model comparison.
- `statistical_tests.py` — Kupiec POF, Christoffersen independence, combined validation verdict.

8.6 `src/riskforge/stress/`

- `scenarios.py` — applies stress shocks; also builds scenario sets, sensitivity analysis, and a historical worst-day scenario.
- `reverse_stress.py` — runs the constrained optimization, verifies the solution independently, and performs a multi-start stability check.

8.7 `src/riskforge/attribution/` and `reporting/`

- `attribution.py` — variance contribution, scenario contribution, average factor ranking.
- `reporting/report.py` — saves CSV outputs, figures, and the validation report, making results reproducible and easy for the dashboard to consume.

8.8 `tests/ (36 tests)` and `dashboard/app.py`

The test suite covers data quality, portfolio math, VaR/ES formulas, backtesting behaviour, stress scenarios, reverse-stress feasibility, and attribution — combining formula checks with small manual examples. `dashboard/app.py` is the Streamlit presentation layer: it reads result CSVs and never implements the core VaR formulas itself, which is intentional (see Section 4.2).

INTERVIEW ANGLE

If asked “walk me through your codebase,” use `scripts/run_pipeline.py` as your mental map — it calls every module in Section 8’s table in sequence, in the same order as the architecture diagram in Section 4.

9. Reference Results and Interpretation

9.1 Model comparison

The reference run produced 2,358 forecasts for each model.

Model	Forecasts	Exceptions	Exception rate	Target	Avg VaR	Avg exceedance
Historical VaR	2,358	32	1.357%	1.0%	1.391%	0.376%
Parametric VaR	2,358	31	1.315%	1.0%	1.319%	0.405%
EWMA VaR	2,358	37	1.569%	1.0%	1.274%	0.339%

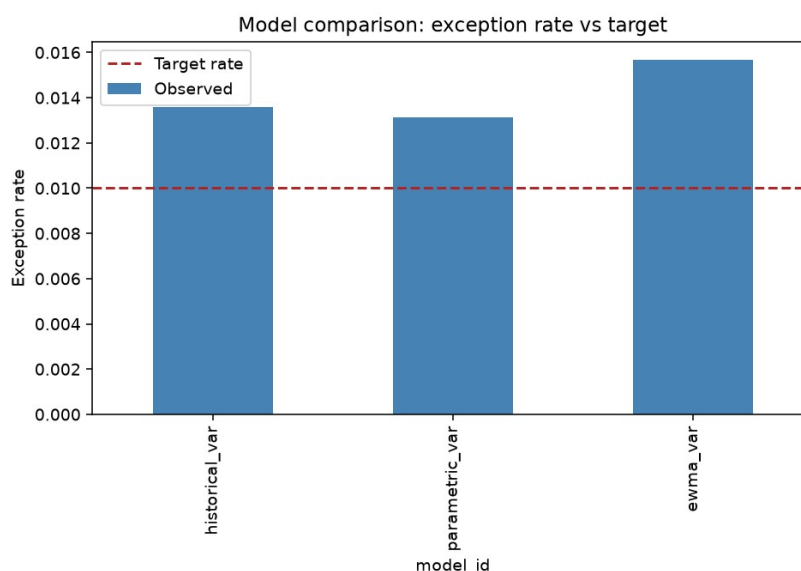


Figure 9.1 — Model comparison: exception rate vs. 1% target across the three VaR models.

Parametric VaR is numerically closest to the 1% target exception rate — but “closest exception rate” does not automatically mean “best model.” The statistical tests below matter just as much.

9.2 Kupiec and Christoffersen results

Model	Kupiec p-value	Kupiec result	Christoffersen p-value	Independence result	Overall
Historical VaR	0.0984	Pass	0.0766	Pass	Pass
Parametric VaR	0.1429	Pass	0.0673	Pass	Pass
EWMA VaR	0.0103	Review	0.1362	Pass	Review

At the 5% significance level: Historical and Parametric VaR do not provide enough evidence to reject the target exception rate; EWMA does provide evidence against the target exception-rate assumption. None of the three models shows a Christoffersen independence rejection at 5%.

9.3 What EWMA's Kupiec failure actually means

EWMA's exception rate is 1.569% against a 1.0% target, with a Kupiec p-value of 0.0103. Since $0.0103 < 0.05$, the project rejects the Kupiec null hypothesis for EWMA in this reference run.

INTERVIEW ANGLE

Never say “EWMA is useless.” Say: “EWMA did not pass unconditional coverage in this run, so its calibration should be reviewed — I'd investigate the lambda value, the window length, and the normal-tail assumption before concluding anything stronger.”

9.4 Backtest visuals

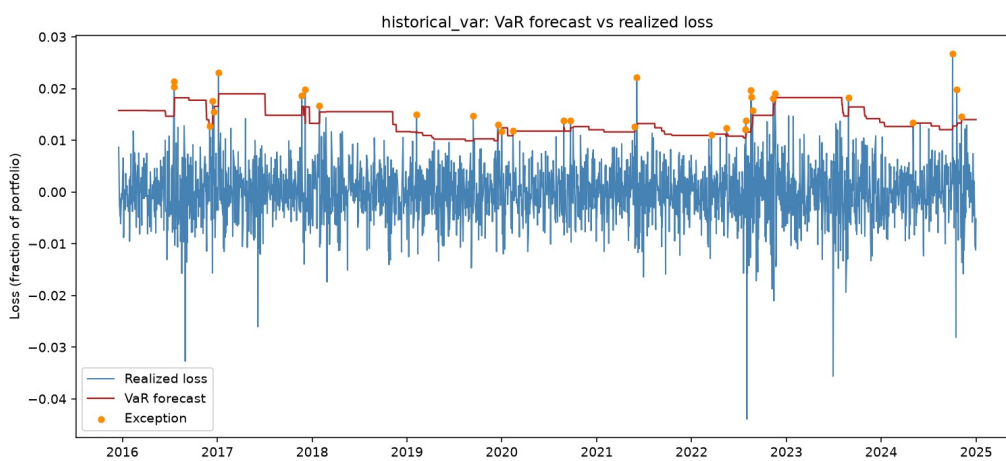


Figure 9.2 — Historical VaR backtest: realised loss vs. VaR forecast, with exceptions marked.

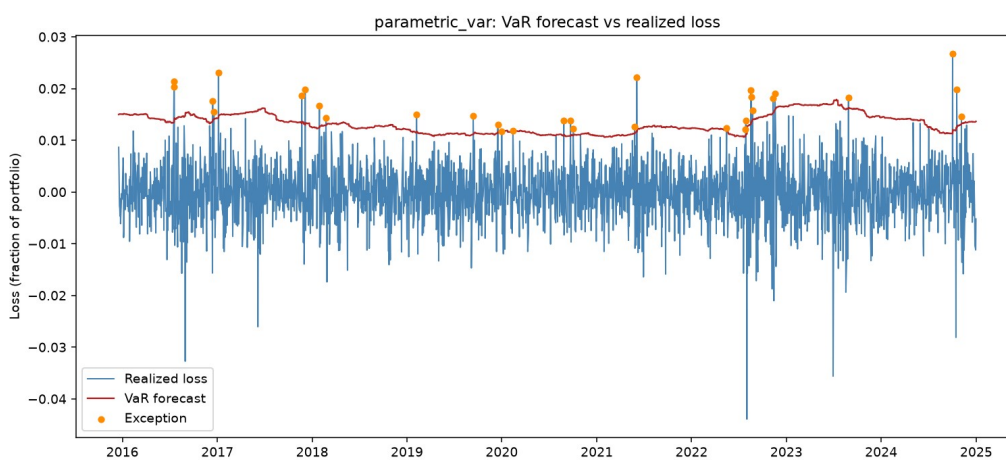


Figure 9.3 — Parametric VaR backtest.

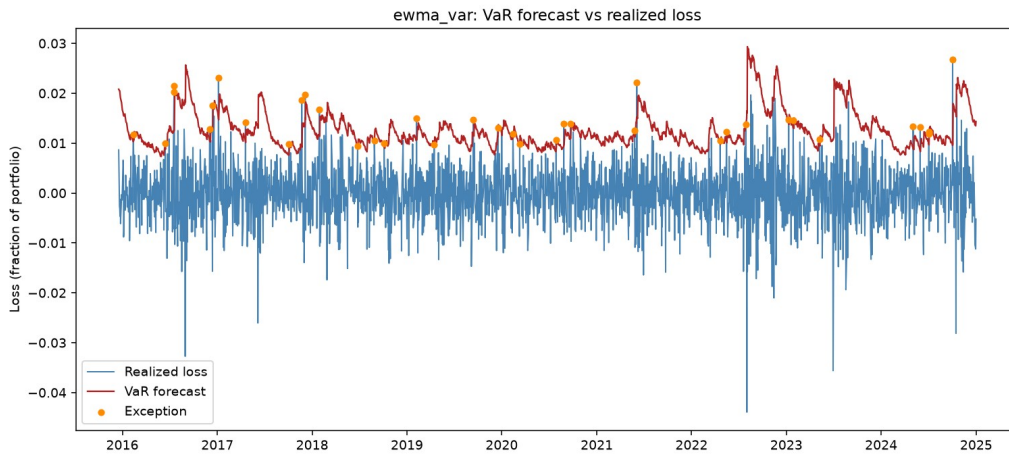


Figure 9.4 — EWMA VaR backtest, showing the clustering of exceptions the Christoffersen test evaluates.

9.5 Stress test results

Scenario	Portfolio loss
EQUITY_CRASH_2008_STYLE	10.65%
FX_CRISIS	4.00%
RATES_SHOCK_UP	3.65%
OIL_SUPPLY_SHOCK	-1.25%

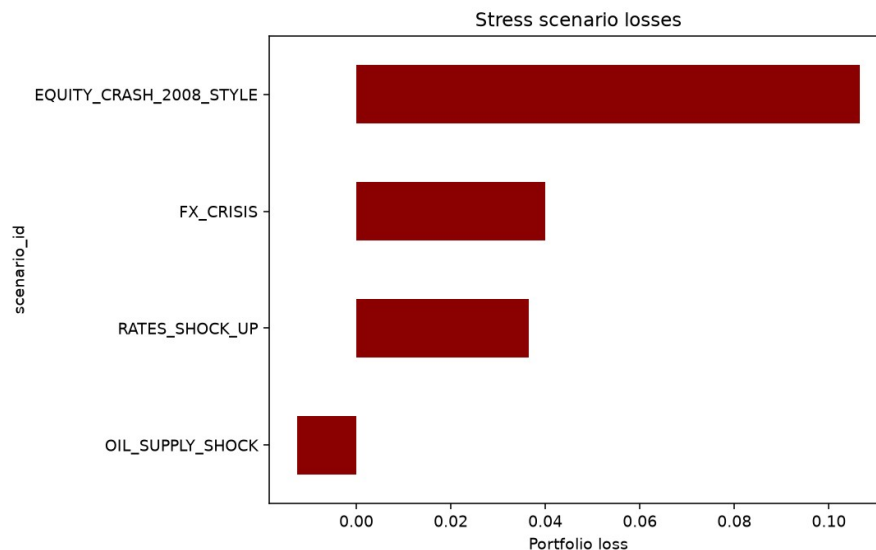


Figure 9.5 — Portfolio loss across the four configured stress scenarios.

The equity-crash scenario is the worst configured scenario. The oil-supply scenario actually produces a negative portfolio loss under the project's linear mapping — meaning the combined shocks generate a net portfolio gain rather than a loss. This is a useful reminder that a “stress scenario” is not automatically harmful to every portfolio; it depends entirely on the portfolio's factor weights and the direction of each shock.

9.6 Equity sensitivity

Equity shock	Portfolio loss
-30%	13.5%
-20%	9.0%
-10%	4.5%
0%	0.0%
+10%	-4.5%

This is exactly consistent with the 45% equity weight: loss = $-0.45 \times$ equity shock. For a -20% shock: $-0.45 \times -20\% = 9\%$ loss.

9.7 Reverse stress result

Target: 10% portfolio loss.

Factor	Required shock
EQUITY_INDEX	-18.367%
RATES_10Y	0.000%
FX_USD	-6.122%
OIL	-4.082%
GOLD	-4.082%

Resulting loss: 10.0%. Constraint status: SATISFIED. L2 norm: approximately 0.202031. Five different optimisation starts produced essentially the same L2 norm and shock vector — evidence that the numerical solution is stable for this problem, though not proof of a universal unique solution in a real market model.

9.8 Risk attribution

Factor	Portfolio weight	Variance share
EQUITY_INDEX	45%	75.07%
OIL	10%	22.59%
FX_USD	15%	3.57%
GOLD	10%	-0.60%
RATES_10Y	20%	-0.63%

Equity has 45% weight but explains about 75% of portfolio variance, because contribution depends on both the portfolio weight and the covariance structure — it is wrong to assume a 45% weight means 45% of risk.

Factor	Average absolute scenario contribution
EQUITY_INDEX	5.0625%
OIL	2.0000%
FX_USD	0.7500%
GOLD	0.6000%

Factor	Average absolute scenario contribution
RATES_10Y	0.2000%

Equity again dominates — creating a consistent story across variance attribution, stress scenarios, and reverse stress.

RESEARCH NOTE

The strongest result in this project is not any single number. It is the chain: model → validation → stress → reverse stress → attribution. That chain is what turns the project from a simple VaR calculation into a complete risk-modelling workflow — and it is the framing to lead with when describing the project's value.

10. Streamlit Dashboard Deep Dive

10.1 What the dashboard does

The dashboard finds the project root, finds the results/ directory, reads result CSVs, reads YAML configuration, displays metrics, creates Plotly charts, lets the user switch pages, and shows methodology and assumptions. The core model calculations remain entirely in `src/riskforge/` — the dashboard only reads and presents.

10.2 The seven pages

Page	Purpose
1. Project Story	High-level explanation: closest VaR model to target, models passing validation, worst stress scenario, largest variance share; the project as six steps
2. VaR Model Lab	Explains and compares Historical / Parametric / EWMA VaR; a model scoreboard and an interactive explorer showing realised loss vs. VaR forecast with exceptions marked
3. Backtesting	PASS/REVIEW card per model, exception rates, Kupiec and Christoffersen explanations, and the overall validation verdict
4. Stress Testing	Scenario leaderboard, scenario losses, a factor-contribution heatmap, and the equity sensitivity curve
5. Reverse Stress	Target loss, solved loss, constraint status, L2 norm, required shock by factor, and multi-start stability results
6. Risk Attribution	Variance contributions (equity $\approx$ 75.1%) and scenario contribution ranking
7. Methodology	Horizon, confidence, rolling window, EWMA lambda, and plain definitions of every concept used, plus explicit assumptions

10.3 Design principles

The dashboard deliberately avoids huge data tables as the main story, unexplained charts, overly technical controls, and unnecessary machine-learning components. Instead it uses metric cards, explanation cards, interactive charts, scenario visuals, and plain-English takeaways. The goal: a person should understand what the project does by using the dashboard, even before opening the code.

10.4 A real engineering lesson from this project

An earlier version of the dashboard had a bug caused by using `bt.var` instead of `bt["var"]`. In pandas, `bt.var` can silently resolve to the DataFrame's built-in variance method rather than the intended `var` column. The corrected dashboard explicitly uses bracket notation everywhere.

INTERVIEW ANGLE

This is a great small story to tell in an interview about debugging discipline: “I hit a subtle pandas bug where a column named `var` collided with the DataFrame’s `.var()` method. I fixed it by switching to bracket notation everywhere and it taught me to be deliberate about column-name collisions with built-in methods.”

11. Testing, QA and Reproducibility

11.1 Why testing matters in quantitative risk

A formula can be mathematically correct but implemented incorrectly — a wrong sign, a wrong quantile, future data accidentally included, misaligned weights, ignored shock bounds, a reversed exception condition. Risk modelling needs tests specifically to catch these implementation errors, not just logic errors.

11.2 The 36 tests, by group

Test group	What it verifies
Data quality	Duplicates, missing values, injected stale values, long/wide transformations — tests intentionally inject bad data to confirm the checks actually catch it
Portfolio	Weights sum to 1, unknown factors rejected, valid weights accepted, manual calculation matches code, covariance/variance calculation works
VaR / ES	Parametric sign convention and formula from moments, historical VaR and a small hand-calculable example, $ES \geq VaR$ in the test case, rolling output length, EWMA reacts to a shock
Backtesting	Exception flag logic, Kupiec at the target rate and far from it, Christoffersen detects clustering and does not flag scattered exceptions, model comparison output
Stress	Scenario calculation, unknown-factor rejection, scenario row counts, linear sensitivity
Reverse stress	Target loss reached, bounds respected, independent verification agrees, infeasible targets reported, multi-start runs reach a feasible target
Attribution	Variance contributions sum to one, scenario ranking ordered correctly

11.3 Reproducibility

The synthetic generator uses seed 42, so the same configuration and code should reproduce the same reference data and results, subject to environment and package-version differences. The project also stores result artefacts on disk instead of making the dashboard recalculate everything on every page load.

11.4 The no-look-ahead principle

This is one of the most important technical concepts in the project. For a forecast on date t , the VaR model must not use the return from date t . The historical VaR code explicitly uses `returns.iloc[t-window:t]`, which stops before t . The backtest then compares that forecast with the return that actually happened on date t — the correct direction is past data $\rightarrow$ forecast $\rightarrow$ future realised outcome, never the reverse.

11.5 A known, disclosed configuration gap

`base.yaml` contains `min_train_window: 500`, but the current rolling VaR functions actually start after the configured 250-day window — so `min_train_window` is not currently enforced by the pipeline.

INTERVIEW ANGLE

If asked “did you implement a separate 500-day minimum training gate?”, answer: “The configuration contains a 500-day minimum-training parameter, but the current implementation's rolling functions use the 250-day estimation window directly. That's a known cleanup item I'd fix before calling this production-ready.” This kind of honest disclosure is a strength, not a weakness.

12. Limitations and Honest Caveats

This section is extremely important for interview defense — a statistically and financially literate project is as much about restraint as it is about results.

12.1 Synthetic data

The reference results come from synthetic data. Exception rates are not real market statistics, stress losses are not historical market facts, and attribution is not evidence about a real portfolio. The real-data loaders exist precisely so the pipeline can be rerun properly.

12.2 Linear portfolio mapping

The project assumes $R_p = \sum w_i R_i$ — a first-order linear approximation. Real portfolios can contain options, bonds, swaps, structured products, and other nonlinear payoffs, for which a simple weighted-return mapping may be insufficient.

12.3 Simplified rates, normal-assumption, and EWMA calibration

- RATES_10Y is represented as a raw yield change rather than being translated into price/P&L through a proper duration or DV01 sensitivity framework.
- Parametric VaR assumes normal returns; real financial returns can show fat tails, skewness, and volatility clustering — part of why comparing multiple models is useful in the first place.
- EWMA uses $\lambda = 0.94$ and a normal tail for VaR; the project does not perform a full lambda calibration study, so responsiveness at other lambda values is unexplored.

12.4 Fixed rolling window and ES reporting gap

The main VaR models use a 250-observation rolling window — a design choice; a different window would change the results. Historical ES is implemented and calculated in the pipeline, but there is no dedicated ES CSV output and the dashboard does not show a dedicated ES result card or chart. ES is therefore part of the model layer but not as fully surfaced as VaR — a clear, low-effort future improvement.

12.5 SQL is not live

The PostgreSQL schema exists, but the current pipeline does not write its results into PostgreSQL. Never claim the project uses a production SQL database — the accurate statement is: “I designed a PostgreSQL schema for future integration.”

12.6 What the reverse-stress and statistical results do not prove

- The reverse-stress solution is the minimum-norm feasible shock under the chosen linear model and bounds — not the most likely scenario, not the worst imaginable scenario, and not a prediction of a future crisis.
- A Kupiec or Christoffersen p-value above 0.05 does not prove a model is correct; it only means the test did not provide enough evidence, at that significance level, to reject the null hypothesis.
- Stress-scenario results depend entirely on the shocks written into configs/stress_scenarios.yaml — changing those assumptions changes every downstream number.

12.7 What you should never claim about this project

- Basel regulatory compliance
- A production bank risk engine
- Real-market predictive accuracy
- Regulatory capital approval
- Monte Carlo simulation
- Counterparty credit exposure, PFE, or EAD
- Real portfolio losses

12.8 What would come next in a production-grade version

1. Real market data feeds and proper market-data validation.
2. Richer instrument-level P&L, including fixed-income duration/DV01 mapping and nonlinear pricing.
3. Multiple portfolios and live SQL persistence.
4. Versioned model configurations and automated monitoring.
5. Model performance drift alerts and stronger scenario calibration.
6. Confidence intervals and more formal model comparison.
7. More complete Expected Shortfall reporting.
8. Stronger CI/CD and deployment controls.

13. Interview Defense Guide

60-second pitch

"RiskForge-QRM is an educational quantitative market-risk modelling and validation framework. I built a fixed-weight portfolio using five market risk factors and estimated one-day 99% VaR using three approaches: Historical, Parametric and EWMA. I did not stop at calculating VaR; I backtested the forecasts using exception rates, Kupiec and Christoffersen tests. I then added stress testing to study severe predefined scenarios, reverse stress testing to find the shocks required to reach a 10% portfolio loss, and risk attribution to identify which factors drive portfolio variance. The pipeline is separated into data, portfolio, model, validation, stress and reporting modules, with 36 unit tests and a Streamlit dashboard for interactive interpretation. The reference run uses synthetic data, so I treat the numbers as a reproducible modelling demonstration rather than real market conclusions."

Q: What problem does your project solve?

A: *"It creates a structured way to estimate, validate and explain short-horizon portfolio market risk."*

Q: Why did you choose VaR?

A: *"VaR is a simple, widely understandable way to summarize downside risk as a loss threshold at a chosen confidence level."*

Q: Why 99% confidence?

A: *"It focuses on a severe tail of the loss distribution and gives a 1% expected exception rate — easy to explain and easy to validate."*

Q: Why three VaR models instead of one?

A: *"Each model makes different assumptions. Comparing them shows how much the model choice itself affects the risk estimate."*

Q: Difference between Historical and Parametric VaR?

A: *"Historical VaR uses the empirical return distribution directly. Parametric VaR assumes a distribution — here, normal — and summarizes it with mean and volatility."*

Q: Why EWMA specifically?

A: *"It lets volatility react to recent market movements instead of treating every observation in the rolling window equally."*

Q: What does $\lambda = 0.94$ control?

A: *"It controls how quickly the volatility estimate forgets old information. A higher λ makes the estimate smoother and slower to react."*

Q: What is an exception?

A: *"When realised loss is greater than the VaR forecast for that day."*

Q: Why is backtesting necessary?

A: *"A model can produce a confident-looking number without being well calibrated. Backtesting checks whether its historical forecasts actually behaved reasonably."*

Q: What does the Kupiec test check?

A: “Whether the observed exception frequency is statistically consistent with the target exception probability.”

Q: What does the Christoffersen test check?

A: “Whether exceptions are independent over time, rather than clustering together.”

Q: Which model performed best?

A: “Parametric VaR had the closest exception rate to 1% and passed both statistical tests in this reference run. Historical also passed both. EWMA failed unconditional coverage. I'd say Parametric had the strongest reference-run behaviour, but I'd treat that as evidence from this dataset and setup — not a universal claim that Parametric is always best.”

Q: Why did EWMA fail?

A: “Its exception rate was 1.569% against a 1% target, and its Kupiec p-value was 0.0103 — below the 0.05 threshold — so the observed exception frequency was significantly different from target in this run.”

Q: Does EWMA failing mean the model is bad?

A: “No. It means the current specification didn't pass the chosen coverage test in this run. I'd investigate calibration, lambda, window length, and the distributional assumption before concluding anything stronger.”

Q: What is stress testing?

A: “Applying severe predefined shocks and measuring the resulting portfolio impact.”

Q: What is reverse stress testing?

A: “Instead of asking what a chosen shock causes, it asks what shocks are sufficient to reach a chosen bad outcome.”

Q: Why minimize the L2 norm in reverse stress?

A: “To find a relatively small overall shock vector that still reaches the target loss, rather than an arbitrarily extreme one.”

Q: What did reverse stress actually find?

A: “A 10% portfolio loss can be reached with approximately equity -18.37%, FX -6.12%, oil -4.08%, gold -4.08%, and rates near 0%, under the configured linear model and bounds.”

Q: Why is equity the biggest risk driver?

A: “It has the largest portfolio weight, and its covariance characteristics make its variance contribution even larger than its raw 45% weight would suggest.”

Q: Why can a risk contribution be negative?

A: “Because covariance and diversification effects can reduce total portfolio variance for a given factor.”

Q: Why did you use synthetic data?

A: “The build environment was offline. Synthetic data made the project reproducible and testable, and the code also contains yfinance and CSV loaders so the same pipeline can be rerun with real data.”

Q: Is this a production risk engine?

A: *"No — it's an educational, industry-style prototype."*

Q: What would you improve first?

A: *"I'd rerun against real market data, add proper instrument-level P&L, improve the fixed-income rate mapping, wire the SQL layer into the pipeline, surface Expected Shortfall in the reporting layer, and strengthen monitoring."*

Q: Is SQL actually used?

A: *"The project contains a PostgreSQL schema, but the current Python pipeline doesn't yet write to a live PostgreSQL database."*

Q: Did you use machine learning?

A: *"No — the core project is deliberately statistical and quantitative rather than machine-learning based."*

Q: What's the strongest engineering part of this project?

A: *"The separation of modelling and presentation, the rolling out-of-sample design with no look-ahead bias, the unit tests, reproducible outputs, and explicit, disclosed assumptions."*

Q: What's the biggest weakness?

A: *"The reference run uses synthetic data and a simplified linear risk-factor mapping. I would not present it as a production risk engine."*

Q: What is look-ahead bias, and how did you avoid it?

A: *"Look-ahead bias is using future information in a forecast that's supposed to represent what could have been known at the time. For each date, my rolling model only uses observations strictly before that date, and the forecast is then compared against the return that actually happened on that date."*

Q: What did you learn from this project overall?

A: *"The main lesson is that quantitative modelling isn't just about calculating a formula correctly. You need data quality checks, explicit assumptions, formal validation, stress analysis, explainability, and testing built around the formula — the formula is the easy part."*

14. Glossary

Term	Plain-English meaning
Risk factor	A market variable that can affect portfolio value (e.g. equity index, oil price, FX rate).
Portfolio	A collection of exposures or assets.
Return	Percentage change in value.
Volatility	A measure of how much returns move around.
Covariance	A measure of how two variables move together.
Quantile	A value that separates a certain percentage of observations.
Tail	The extreme part of a distribution.
VaR	A loss threshold at a chosen confidence level.
Expected Shortfall	Average loss beyond the VaR threshold.
Exception	A realised loss larger than the predicted VaR.
Backtesting	Comparing model forecasts with realised outcomes.
p-value	A measure used in hypothesis testing to assess how compatible the data is with the null hypothesis.
Significance level	The cutoff used to decide whether to reject a null hypothesis; here, 5%.
Stress test	Applying a severe predefined scenario.
Reverse stress test	Finding what shocks could cause a chosen bad outcome.
Attribution	Breaking total risk into factor contributions.
EWMA	Exponentially Weighted Moving Average.
Model validation	Checking whether a model behaves reasonably under defined tests.
Look-ahead bias	Accidentally using future information when creating a historical forecast.
Synthetic data	Artificially generated data designed to resemble certain statistical properties of real data.
Fat tails	More extreme observations than a normal distribution would usually produce.
Volatility clustering	High-volatility periods tend to be followed by more high-volatility periods.
Linear mapping	A model where the output changes proportionally with input changes.
L2 norm	A measure of the overall size of a vector: $\ x\ _2 = \sqrt{\sum x_i^2}$.

15. Quick Revision Sheet

Project

RiskForge-QRM — fixed-weight, five-factor market-risk portfolio; 2015–2024 synthetic data; 13,045 raw price rows; 2,358 VaR forecasts per model.

Core workflow

Data quality → portfolio construction → Historical / Parametric / EWMA VaR → backtesting (Kupiec, Christoffersen) → stress testing → reverse stress → risk attribution → validation report → dashboard.

Numbers to have ready

Result	Value
Historical VaR exception rate	1.357% (target 1.0%), Kupiec p=0.0984 Pass
Parametric VaR exception rate	1.315% (target 1.0%), Kupiec p=0.1429 Pass
EWMA VaR exception rate	1.569% (target 1.0%), Kupiec p=0.0103 Review
Christoffersen (all three)	Pass — no clustering detected at 5%
Worst stress scenario	EQUITY_CRASH_2008_STYLE, 10.65% loss
Best (mildest) stress scenario	OIL_SUPPLY_SHOCK, -1.25% (a net gain)
Reverse-stress target	10% loss, solved exactly, L2 norm ≈ 0.202031
Reverse-stress shocks	Equity -18.37%, FX -6.12%, Oil -4.08%, Gold -4.08%, Rates 0%
Equity variance share	75.07% (vs. 45% portfolio weight)
Automated tests	36, covering data/portfolio/models/validation/stress/attribution
Random seed	42 (reproducibility)

Three things to never forget

1. The reference results are from synthetic data — never present them as real market findings.
2. VaR is a threshold, not the worst possible loss, and a passing backtest is not proof a model is correct.
3. This is a market-risk framework, not a credit-exposure (PFE/EAD) or production regulatory-capital system.

Best one-line project explanation

"I built a fixed-weight market-risk portfolio, estimated one-day 99% VaR using three statistical approaches, backtested those forecasts, stress-tested the portfolio, solved a reverse-stress problem, and attributed portfolio risk back to individual risk factors."

16. Conclusion and Future Work

RiskForge-QRM demonstrates a complete, disciplined quantitative market-risk workflow: from data quality and portfolio construction, through three competing VaR models with rolling out-of-sample forecasting and no look-ahead bias, into formal statistical backtesting (Kupiec, Christoffersen), scenario and reverse-stress analysis, and factor-level risk attribution — all wrapped in a modular, tested codebase and a clear presentation-layer dashboard.

The consistent thread throughout is honesty about scope: every strong result (a passing backtest, a dominant risk factor, a solved reverse-stress problem) is paired with a clear statement of what it does and does not prove, and every known gap — synthetic data, the unwired SQL layer, the unenforced `min_train_window`, the unsurfaced ES output — is documented rather than hidden. That combination of technical breadth across a full model-validation lifecycle, plus the discipline to state limitations clearly, is exactly what this report is designed to demonstrate.

Suggested future extensions

- Rerun the full pipeline against real market data via the existing yfinance or CSV loaders.
- Add instrument-level P&L and a proper duration/DV01 mapping for the rates factor.
- Wire the existing PostgreSQL schema into the live pipeline for persistent, queryable results.
- Surface Expected Shortfall as a first-class output alongside VaR in both the tables and the dashboard.
- Run a lambda-calibration study for EWMA and investigate why its Kupiec test failed in this reference run.
- Extend risk attribution and reverse stress to multiple portfolios and nonlinear instruments.